Question 1

(i) Compare and contrast structured programming and object-oriented programming.

Structured Programming focuses on dividing a program into smaller functions or procedures, following a **top-down approach**. **Object-Oriented Programming (OOP)** organizes software design around objects (data and behavior) rather than functions, following a **bottom-up approach**.

Key Differences:

Feature	Structured Programming	Object-Oriented Programming (OOP)
Approach	Top-down approach.	Bottom-up approach.
Primary Focus	Focuses on functions and logic execution.	Focuses on data security and objects.
Data Movement	Data moves freely throughout the program from function to function.	Data is encapsulated within objects and accessed via methods.
Access Specifiers	No access specifiers; data security is minimal.	Uses specifiers (private, public, protected) for high security.
Code Reusability	Limited reusability (via function calls).	High reusability (via inheritance).
Key Concepts	Functions, structures, modules.	Encapsulation, Abstraction, Inheritance, Polymorphism.
Examples	C, Pascal.	Java, C++, Python.

(ii) Describe the concept of type casting and type conversion.

Type conversion and type casting deal with changing a variable from one primitive data type to another in Java.

1. Type Conversion (Implicit / Widening)

- **Definition:** Converting a lower data type (smaller size) to a higher data type (larger size) automatically by the Java compiler without data loss.
- **Condition:** The two types must be compatible, and the destination type must be larger than the source type (byte \rightarrow short \rightarrow int \rightarrow long \rightarrow float \rightarrow double).
- **Example:**

```
int num = 100;
double doubleNum = num; // Implicit conversion from int to double
(100.0)
```

2. Type Casting (Explicit / Narrowing)

- **Definition:** Converting a higher data type (larger size) to a lower data type (smaller size) manually by the programmer.
- **Characteristics:** Requires an explicit cast operator (datatype) and may result in a loss of data or precision.
- **Example:**

```
double doubleVal = 9.78;
int intVal = (int) doubleVal; // Explicit casting (truncates
decimal part to 9)
```

Question 2

(i) Explain the different access specifiers in Java.

Access specifiers (or access modifiers) control the scope and visibility of classes, constructors, methods, and variables in Java. There are four types:

1. **private:**
   - Accessible **only within the same class**.
   - Provides the highest level of data hiding (used for encapsulation).
2. **default (package-private):**
   - Applied when no modifier is specified.
   - Accessible **only within the same package**.
3. **protected:**
   - Accessible **within the same package**, and by **subclasses (child classes) in different packages** through inheritance.
4. **public:**
   - Accessible **from anywhere** in the application (any package or class).

Visibility Matrix:

Access Modifier	Same Class	Same Package	Subclass (Diff Package)	Outside Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

(ii) Write a program in Java to print the factorial of a number.

```
import java.util.Scanner;

public class Factorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter a non-negative integer: ");
        int num = sc.nextInt();

        if (num < 0) {
            System.out.println("Factorial is not defined for negative
numbers.");
        } else {
            long factorial = 1;
            for (int i = 1; i <= num; i++) {
                factorial *= i;
            }
            System.out.println("Factorial of " + num + " = " +
factorial);
        }

        sc.close();
    }
}
```

Question 3

(i) Elaborate on the concept of the String class with any 4 of its methods.

In Java, strings are objects represented by the java.lang.String class. A string is an array of characters enclosed in double quotes.

Key Characteristics:

- **Immutability:** Once created, a String object cannot be changed or modified. Any modification creates a new String object.
- **Storage:** Strings can be created using string literals (stored in the String Constant Pool) or using the new keyword (stored in the Heap memory).

4 Commonly Used Methods:

1. **length()**
    - **Purpose:** Returns the total number of characters in the string.
    - **Syntax:** int len = str.length();
    - **Example:** "Java".length() returns 4.
2. **charAt(int index)**
    - **Purpose:** Returns the character located at the specified zero-based index.
    - **Syntax:** char ch = str.charAt(index);
    - **Example:** "Hello".charAt(1) returns 'e'.
3. **substring(int beginIndex, int endIndex)**
    - **Purpose:** Returns a part of the string from beginIndex up to (but excluding)

endIndex.
    - **Syntax:** String sub = str.substring(start, end);
    - **Example:** "Object".substring(0, 3) returns "Obj".
4. **toUpperCase()**
    - **Purpose:** Converts all characters in the string to upper case.
    - **Syntax:** String upper = str.toUpperCase();
    - **Example:** "java".toUpperCase() returns "JAVA".

(ii) Write a program in Java to find and display the transpose of a 2D matrix with user input.

```
import java.util.Scanner;

public class MatrixTranspose {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();

        int[][] matrix = new int[rows][cols];
        int[][] transpose = new int[cols][rows];

        // Reading matrix elements
        System.out.println("Enter matrix elements:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        // Computing transpose
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                transpose[j][i] = matrix[i][j];
            }
        }

        // Displaying Original Matrix
        System.out.println("\nOriginal Matrix:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
```

```
        }

        // Displaying Transposed Matrix
        System.out.println("\nTransposed Matrix:");
        for (int i = 0; i < cols; i++) {
            for (int j = 0; j < rows; j++) {
                System.out.print(transpose[i][j] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```